

SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
CHENNAI – 602105
Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – DATA INTERPRETATION

Course Code:	DSA05	Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO1	Assessment:	Tool 2 – Schema Analysis and Viva Voce
Weightage:		Total Marks:	20 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BTL6)			
Student Name:	G Rajasekhar Reddy	Reg. No.:	192424216
Section / Batch:	BTech-AI&DS	Date of Submission:	20-07-2026
Faculty In-charge:	Dr. B. Shyamala Gowri	Project Mode:	Individual Submission

Code of Conduct

I G .Rajasekhar Reddy(192424216) (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: G. Rajasekhar Reddy

Requirement Analysis (4 Marks)	ER Diagram Design (4 Marks)	Table Design & Normalization (4 Marks)	Database Connectivity using Python (4 Marks)	CRUD Operations (4 Marks)	Total (20 Marks)

--	--	--	--	--	--

19. Weather Data Analysis

Files:

- ☐ Weather.csv
- ☐ Stations.csv
- ☐ Weather.json
- ☐ Weather.xml

Read weather datasets from CSV, JSON, and XML. Merge weather and station data, calculate monthly average temperature, identify extreme weather conditions, summarize seasonal rainfall, and export the cleaned dataset.

Program:

```
import pandas as pd
```

```
weather_csv = pd.read_csv("Weather.csv")
```

```
stations = pd.read_csv("Stations.csv")
```

```
weather_json = pd.read_json("Weather.json")
```

```
weather_xml = pd.read_xml("Weather.xml")
```

```
weather = pd.concat([weather_csv, weather_json, weather_xml], ignore_index=True)
```

```
merged = pd.merge(weather, stations, on="Station_ID")
```

```
merged["Date"] = pd.to_datetime(merged["Date"])
```

```
merged["Month"] = merged["Date"].dt.month
```

```
monthly_avg = merged.groupby("Month")["Temperature"].mean()
```

```
extreme_weather = merged[(merged["Temperature"] > 40) | (merged["Temperature"] < 5)]
```

```
def season(month):
```

```
    if month in [12, 1, 2]:
```

```
        return "Winter"
```

```
    elif month in [3, 4, 5]:
```

```
        return "Summer"
```

```
    elif month in [6, 7, 8, 9]:
```

```
        return "Monsoon"
```

```
    else:
```

```
        return "Post-Monsoon"
```

```
merged["Season"] = merged["Month"].apply(season)
```

```
seasonal_rainfall = merged.groupby("Season")["Rainfall"].sum()
```

```
merged.to_csv("Cleaned_Weather.csv", index=False)
```

```
print("Monthly Average Temperature")
```

```
print(monthly_avg)
```

```
print("\nExtreme Weather")
```

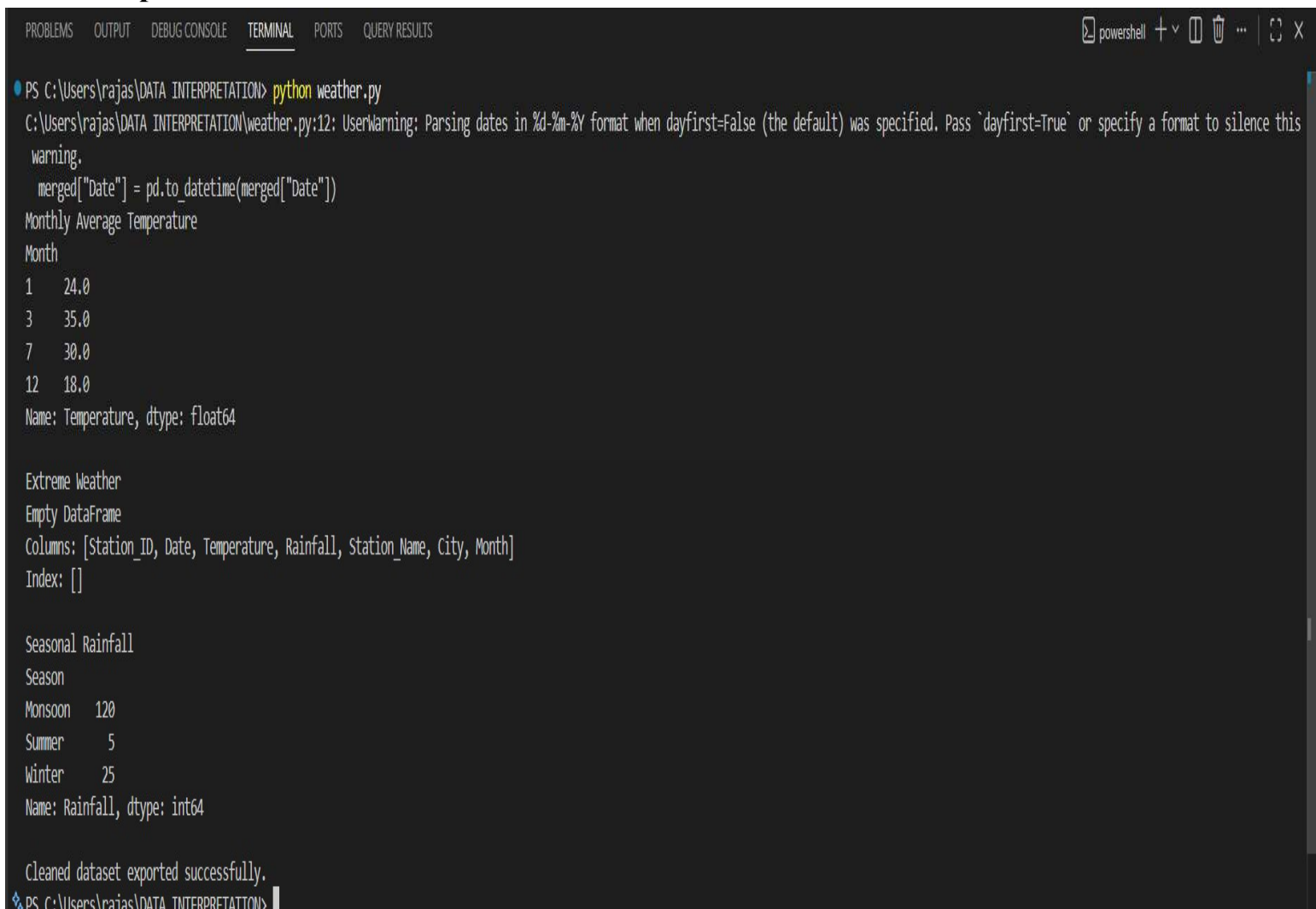
```
print(extreme_weather)
```

```
print("\nSeasonal Rainfall")
```

```
print(seasonal_rainfall)
```

```
print("\nCleaned dataset exported successfully.")
```

Output:



```
PS C:\Users\rajas\DATA INTERPRETATION> python weather.py
C:\Users\rajas\DATA INTERPRETATION\weather.py:12: UserWarning: Parsing dates in %d-%m-%Y format when dayfirst=False (the default) was specified. Pass `dayfirst=True` or specify a format to silence this warning.
  merged["Date"] = pd.to_datetime(merged["Date"])
Monthly Average Temperature
Month
1      24.0
3      35.0
7      30.0
12     18.0
Name: Temperature, dtype: float64

Extreme Weather
Empty DataFrame
Columns: [Station_ID, Date, Temperature, Rainfall, Station_Name, City, Month]
Index: []

Seasonal Rainfall
Season
Monsoon    120
Summer      5
Winter     25
Name: Rainfall, dtype: int64

Cleaned dataset exported successfully.
PS C:\Users\rajas\DATA INTERPRETATION>
```

20. COVID-19 Vaccination Analysis

Files:

- ☐ Vaccination.csv
- ☐ Districts.csv
- ☐ Vaccination.json
- ☐ Vaccination.xml

Read vaccination datasets from multiple formats. Merge vaccination and district records, calculate vaccination percentage, identify low-coverage districts, summarize vaccine usage, and export the cleaned dataset.

Program:

```
import pandas as pd
```

```
vaccination_csv = pd.read_csv("Vaccination.csv")
districts = pd.read_csv("Districts.csv")
vaccination_json = pd.read_json("Vaccination.json")
vaccination_xml = pd.read_xml("Vaccination.xml")

vaccination = pd.concat([vaccination_csv, vaccination_json, vaccination_xml], ignore_index=True)

merged = pd.merge(vaccination, districts, on="District_ID")

merged["Vaccination_Percentage"] = (merged["Vaccinated"] / merged["Population"]) * 100

low_coverage = merged[merged["Vaccination_Percentage"] < 70]

vaccine_usage = merged.groupby("Vaccine")["Vaccinated"].sum()

merged.to_csv("Cleaned_Vaccination.csv", index=False)

print("Vaccination Percentage")
print(merged[["District_Name", "Vaccination_Percentage"]])

print("\nLow Coverage Districts")
print(low_coverage[["District_Name", "Vaccination_Percentage"]])

print("\nVaccine Usage")
print(vaccine_usage)

print("\nCleaned dataset exported successfully.")
```

Output:

PS C:\Users\rajas\DATA INTERPRETATION> python vaccination.py

Vaccination Percentage

	District_Name	Vaccination_Percentage
0	Chennai	80.0
1	Hyderabad	50.0
2	Bangalore	95.0

Low Coverage Districts

	District_Name	Vaccination_Percentage
1	Hyderabad	50.0

Vaccine Usage

Vaccine

Covaxin 50000

Covishield 175000

Name: Vaccinated, dtype: int64

Cleaned dataset exported successfully.

PS C:\Users\rajas\DATA INTERPRETATION>